

Manual de rol — Persona C

Interacción, Monetización e Infraestructura

Cresco · Versión 1.2 · 8 de agosto de 2026 · Envío del equipo: 28 de septiembre

Corrige la v1.0 del 3 de agosto de 2026. Ver <docs/99-archivo/ERRATA-2026-08.md> para el detalle de cada cambio. Esta versión 1.2 reinicia el calendario (tu semana 1 empieza el 9 de agosto) y te asigna una tarea nueva el día 1: scaffoldear `servidor/` y `movil/`.

Tu rol en una frase

Tú sostienes todo lo que rodea a la app — y eres dueño del único requisito que puede descalificarnos.

RevenueCat es obligatorio para el Shipaton. Si no funciona en un dispositivo real, no importa qué tan buena esté la app: no calificamos. Ese riesgo es tuyo y por eso arranca en la semana 1, no al final.

Tu rol es el más heterogéneo de los tres: pagos, notificaciones, citas, alertas, integración continua y despliegue. **A diferencia de la v1.0 de este manual, ya no dependes de los trámites de Google Play:** la categoría Next Gen del Shipaton no exige cuenta de desarrollador ni publicación en tienda. Tu riesgo real sigue siendo que RevenueCat funcione de forma demostrable en un teléfono real.

Parte 1 — Contexto del proyecto

1.1 Qué construimos

App Android de comunicación entre docentes y representantes legales de escuelas del Ecuador, para seguimiento de conducta. El estudiante nunca usa la app.

1.2 El puntaje, en resumen

Cada estudiante parte de 60 puntos por parcial. Positivas +1 o +2, negativas -1 a -3, topes de +4 y -5 por día, piso 0 y techo 100. Ese motor es de Persona B, pero necesitas conocerlo porque tu módulo de inconformidades devuelve puntos cuando una acción se resuelve como modificada.

1.3 Decisiones ya tomadas

#	Decisión
ADR-001	PostgreSQL + Drizzle ORM
ADR-002	Next.js como servidor, Expo para las apps
ADR-003	BetterAuth con plugin de organización
ADR-004	Permisos en dos capas: código + RLS
ADR-005	El puntaje es derivado, nunca un contador
ADR-006	RevenueCat es la fuente de verdad de pagos ← tu ADR
ADR-007	Entrega sin tienda, repositorio abierto bajo AGPL-3.0 ← tu ADR

Parte 2 — RevenueCat: tu responsabilidad crítica

2.1 Por qué es lo primero

El Shipaton exige el SDK integrado y funcionando. Es el único requisito que, si falla, invalida todo el trabajo de los tres. Por eso lo pruebas en la semana 1, cuando todavía hay tiempo de reaccionar.

2.2 Lo que debes saber antes de empezar

RevenueCat en Expo no funciona en Expo Go. Necesitas un *development build*, que es una versión de la app compilada con código nativo incluido. Es un paso extra que sorprende a todo el mundo la primera vez.

Ya NO hace falta Google Play Console para el envío. Las reglas oficiales de la categoría Next Gen (verificadas el 7-8 de agosto) dicen textualmente que no se requiere cuenta de desarrollador de Google Play ni publicación en tienda: se evalúa por video de demostración y repositorio de código abierto. Sigue pendiente confirmar en la documentación oficial de RevenueCat qué modo de prueba de compras funciona sin una app publicada, y si `Purchases.getOfferings()` devuelve datos en ese modo. Esa verificación es tu tarea de esta semana.

Si resultara que sí hace falta una app en Play Console para demostrar una compra real, el trámite vuelve a la lista y se arranca ese mismo día: es el único hallazgo que revivería ese riesgo. Avísalo al grupo de inmediato si pasa.

Verifica los requisitos vigentes en la documentación oficial de RevenueCat y de las reglas del Shipaton ([Contexto/reglas-shipaton-next-gen.md](#)): cambian con frecuencia y la información de este manual puede haber quedado desactualizada.

2.3 Los cinco planes

Código	Audiencia	Periodicidad	Qué da
REP_FREE	Representante	Perpetuo	2 reportes anteriores, con publicidad
REP_PREMIUM_MENSUAL	Representante	Mensual	Sin publicidad, 7 reportes, PDF libre
REP_PREMIUM_BIMESTRAL	Representante	Bimestral	Igual — es el "por parcial"

DOC_FREE	Docente	Perpetuo	1 curso activo, 40 estudiantes
DOC_PRO	Docente	Mensual	5 cursos activos, 60 estudiantes

Nota: "por parcial" se mapea a una suscripción bimestral porque Google Play no tiene un ciclo de 6 semanas.

2.4 Reglas del módulo de pagos

- **RevenueCat es la fuente de verdad.** La tabla `suscripcion` es solo una proyección local para consultas rápidas.
- **El webhook es idempotente.** `evento_revenuecat.evento_id_externo` tiene índice único. Un reenvío no puede duplicar una suscripción.
- **Los límites se verifican en el servidor**, nunca solo en la app. Un cliente modificado no debe poder crear un segundo curso.
- **Los límites viven en `plan.limite` como JSON.** Cambiar un límite es un `UPDATE`, no un despliegue.
- Cuando se alcanza un límite, el endpoint devuelve `402` con código `LIMITE_PLAN`, y la app muestra el paywall en ese momento exacto.

2.5 Licencia de código abierto

Next Gen exige repositorio público con `LICENSE` en la raíz antes de enviar. El equipo eligió **AGPL-3.0**, con posibilidad de licenciamiento comercial en paralelo para colegios o clientes, ya que Neofix es titular único del copyright (decisiones L1 y L2).

El archivo ya está en la raíz, con el texto canónico de la FSF sin modificar. Lo que te queda es de semana 6: hacer público el repositorio y **confirmar que GitHub muestra "AGPL-3.0" en la sección About**. Si no aparece, es que el archivo se renombró o se editó — GitHub solo lo reconoce si es texto de licencia íntegro y se llama `LICENSE`.

No edites `LICENSE`. El aviso de copyright de Neofix y la nota de licenciamiento comercial van en el `README.md`, no dentro del texto legal. Tocar el texto rompe la detección automática y, con ella, un requisito de entrega.

Parte 3 — Tu territorio

3.1 Tus tablas

Archivo: `db/schema/interaccion.ts`

`disponibilidad_docente` · `cita` · `inconformidad` · `alerta_emergencia` · `entrega_alerta` · `plan` · `suscripcion` · `evento_revenuecat` · `desbloqueo_recompensado` · `dispositivo` · `notificacion` · `auditoria`

3.2 Tus pantallas

#	Pantalla	App	Semana
P1	Registro del representante	Representante	2
P11	Paywall y suscripción	Representante	2
D19	Paywall del docente	Docente	3
D17	Alerta de emergencia	Docente	4
P10	Alertas recibidas	Representante	4
D15	Bandeja de inconformidades	Docente	5
D16	Horario y citas	Docente	5
P7	Detalle de acción e inconformidad	Representante	5
P8	Agendar cita	Representante	5
P9, P12	Profesor a cargo, ajustes	Representante	opcional

3.3 Tus endpoints

```
POST  /docente/alertas
GET   /docente/inconformidades
POST  /docente/inconformidades/{id}/resolver
GET   /docente/disponibilidad
POST  /docente/disponibilidad
GET   /docente/citas
POST  /docente/citas/{id}/responder
GET   /representante/alertas
```

```
POST /representante/alertas/{id}/confirmar
POST /representante/inconformidades
GET /representante/citas/disponibilidad
POST /representante/citas
GET /suscripcion/estado
POST /webhooks/revenuecat
POST /representante/reporte-acumulado/exportar
POST /dispositivos
DELETE /dispositivos
```

Los endpoints del lado del docente (inconformidades, disponibilidad, citas) y el registro de dispositivo se agregaron al contrato el 8 de agosto — v1.1.0 de `api/openapi.yaml`. Estaban en el backlog pero faltaban en el contrato.

Exportar PDF del acumulado: el endpoint devuelve el archivo directamente en la respuesta, sin URL ni almacenamiento intermedio — así se cumple la regla de que no se guardan archivos en ninguna parte (I3).

3.4 Lo que NO tocas

`db/schema/nucleo.ts` (de A) · `db/schema/conducta.ts` (de B) · las pantallas de registro de curso, estudiantes, acciones o reportes.

Regla crítica: prohibido usar `db.select()` suelto sobre `estudiante`, `matricula`, `accion_registrada` o `reporte_estudiante`. Toda consulta pasa por `db/acceso/`, que construye A.

3.5 Dónde vive tu código — y por qué lo scaffoldas tú

Todos tus endpoints (webhook, rutas de API) viven **dentro de** `servidor/`, el proyecto Next.js. Tus pantallas (D14–D19, P7–P12) viven **dentro de** `movil/`, la única app Expo del proyecto — con enrutado por rol, no dos apps separadas.

Eres tú quien crea las dos carpetas, el **domingo 9, día 1**, junto con el repositorio y el CI (ver Parte 5). Es lógico que sea tuya: eres formalmente dueño de infraestructura, y A necesita `servidor/` ya armado para instalar BetterAuth el martes 11 — dos días de margen, igual que en el plan original.

```
npx create-next-app servidor
npx create-expo-app movil
```

No hace falta ningún paquete compartido ni `npm workspaces` entre las dos. Cada una tiene su propio `package.json`, instalado con su propio `npm install` dentro de su carpeta. El único código realmente compartido sigue siendo `db/schema/` en la raíz, y solo lo importa `servidor/` (el móvil

nunca toca la base de datos directo, solo llama a la API). Esto no reabre ADR-002 — decisiones S1-S6 en Contexto/DECISIONS.md .

Parte 4 — Preparación (semana 1, 4 de agosto en adelante)

4.1 Instalar

Node.js 20+, Git, VS Code.

```
node --version    # v20 o superior
git config --global user.name "Tu Nombre"
git config --global user.email "tu@correo.com"
```

Un teléfono Android real. No es opcional para tu rol: las compras y las notificaciones push no se prueban bien en emulador. Activa las opciones de desarrollador y la depuración por USB.

Cuentas que debes crear:

- RevenueCat (gratuita)
- Expo / EAS (gratuita)
- Devpost, con **correo académico** (requisito de Next Gen) — verifica antes que el dominio de tu universidad esté en la lista JetBrains/swot

Ya no hace falta crear cuenta de Google Play Console. Era el trámite más lento del proyecto en la v1.0 de este manual; Next Gen no lo exige. Si en algún punto se decide publicar de verdad en la tienda (fuera del plan actual), ese trámite se retoma entonces.

4.2 Conceptos que necesitas

¿Qué es un webhook? Una URL de tu servidor que un servicio externo llama cuando pasa algo. RevenueCat llama la tuya cuando alguien compra, renueva o cancela.

¿Qué es idempotencia? Que ejecutar la misma operación dos veces produzca el mismo resultado que ejecutarla una. Si RevenueCat reenvía un evento (y lo hace), tu webhook no debe crear dos suscripciones.

¿Qué es un entitlement? En RevenueCat, un permiso abstracto (por ejemplo `premium`) que varios productos pueden otorgar. Tu app pregunta "¿tiene `premium`?", no "¿compró el producto mensual o el bimestral?".

¿Qué es un development build? Una versión de la app compilada con código nativo incluido. Necesaria porque RevenueCat usa código nativo y Expo Go no lo soporta.

4.3 Clonar

```
git clone <url-del-repo>
cd cresco
npm ci                    # versiones exactas del lockfile
```

Parte 5 — Semana a semana

Semana 1 · 9 al 15 de agosto — Cimientos e infraestructura

Tu semana define si el proyecto califica o no. No la subestimes.

Domingo 9 — Repositorio, CI, y las dos apps

1. Configura la protección de la rama `main` : nadie empuja directo, todo por PR con una aprobación.
2. **Scaffoldea** `servidor/` y `movil/` — tarea nueva desde el 8 de agosto (decisión S6). Ver el detalle en §3.5.

```
npx create-next-app servidor
npx create-expo-app movil
```

A necesita `servidor/` el martes para BetterAuth: hazlo hoy, no lo dejes para después.

3. Monta el CI (GitHub Actions) que corra en cada PR:

```
- npm run typecheck
- npm test
```

Éxito: un PR con un error de tipos no se puede fusionar, y existen `servidor/` y `movil/` con sus propios `package.json`.

Lunes 10 — Development build de Expo

Este es el paso que más problemas da la primera vez. Reserva el día completo.

```
cd movil
npx expo install expo-dev-client
npx eas build --profile development --platform android
```

Instala el resultado en tu teléfono real.

Trampa: si intentas usar Expo Go, RevenueCat fallará con un error de módulo nativo no encontrado. No es un bug tuyo: es que Expo Go no incluye código nativo de terceros.

Éxito: la app abre en tu teléfono desde el development build.

Martes 11 — RevenueCat en sandbox

1. Crea el proyecto en RevenueCat.
2. Instala el SDK: `npx expo install react-native-purchases`
3. Configúralo al arrancar la app:

```
import Purchases from "react-native-purchases";
Purchases.configure({ apiKey: process.env.EXPO_PUBLIC_REVENUECAT_KEY });
```

4. Define el *entitlement* `premium` y los cinco productos.

Éxito de la semana 1: `Purchases.configure()` no lanza error y `Purchases.getOfferings()` devuelve algo en un teléfono real.

Miércoles 12 — Confirmar el modo de prueba sin tienda

En la v1.0 de este manual, este día estaba reservado para Google Play Console. Ya no. En su lugar:

- Confirma en la documentación oficial de RevenueCat qué mecanismo de prueba de compras funciona **sin una app publicada en tienda**, y si `getOfferings()` devuelve datos en ese modo.
- Si resultara que sí hace falta una app en Play Console para una demostración de compra real, el trámite vuelve a la lista ese mismo día.
- Con el tiempo sobrante, adelanta el webhook del fin de semana.

Jueves 13 y viernes 14 — Webhook

Crea `POST /webhooks/revenuecat` dentro de `servidor/`. Debe:

1. Verificar la firma de la petición
2. Insertar en `evento_revenuecat` con el identificador externo
3. Si el identificador ya existe → responder `200` y no hacer nada más
4. Si es nuevo → actualizar `suscripcion` y marcar `procesado_en`

Sábado 15 — Colchón

Sin tarea nueva. Úsalo para lo que se haya corrido, o confirma el criterio de éxito de la semana completa.

Semana 2 · 16 al 22 de agosto — Paywall y notificaciones

Pantalla P1 — Registro del representante

Usa los componentes de BetterAuth. Coordina con A, que ya lo configuró.

Pantalla P11 — Paywall del representante

Muestra los planes disponibles desde `Purchases.getOfferings()`. Nunca escribas los precios a mano: vienen de la tienda.

Consejo: el paywall no debe aparecer al abrir la app. Aparece cuando el representante intenta ver un tercer reporte anterior. El momento importa más que el diseño.

Notificaciones push

```
npx expo install expo-notifications
```

Guarda el token en la tabla `dispositivo`. Prueba enviando una notificación desde el servidor a tu propio teléfono.

Éxito de la semana 2: llega una notificación push a un teléfono real y la compra de prueba en sandbox se completa.

Semana 3 · 23 al 29 de agosto — Límites de plan

Pantalla D19 — Paywall del docente

Se muestra cuando el docente con `DOC_FREE` intenta crear su segundo curso.

Verificación de límites en el servidor

```
export async function verificarLimite(
  userId: string,
  recurso: "cursos_activos" | "estudiantes_por_curso",
  cantidadActual: number,
): Promise<{ permitido: boolean; limite: number }> {
  // lee el plan activo del usuario y compara
}
```

Coordina con A esta semana. Él llama tu función desde su endpoint. Acuerden la firma antes de que cada uno escriba su lado.

Éxito de la semana 3: un docente con `DOC_FREE` recibe `402` al crear su segundo curso, y la app muestra el paywall.

Semana 4 · 30 de agosto al 5 de septiembre — Alertas

Pantalla D17 — Alerta de emergencia

Reglas que debes respetar:

- **Exige reautenticación:** el docente reingresa su contraseña antes de activar. Guardas `reautenticado_en` como constancia.
- **Alcance:** todo el curso o un solo estudiante.
- **Queda auditada:** quién, cuándo, tipo, mensaje, y si fue simulacro.
- **La app debe declarar** que la alerta no sustituye a los servicios de emergencia (ECU 911). Ponlo visible en la pantalla, no escondido en términos.

Consejo: haz que el botón sea difícil de tocar por accidente. Confirmación de dos pasos, no un botón grande en la pantalla principal.

Pantalla P10 — Confirmación de lectura

El representante confirma que la leyó. Se guarda en `entrega_alerta`.

Éxito de la semana 4: una alerta llega a los representantes del curso y la confirmación se registra.

Semana 5 · 6 al 12 de septiembre — Inconformidades y citas

D15, P7 — Inconformidades

- Solo sobre acciones **negativas** vigentes.
- El docente tiene **30 días**; vencido, pasa a `VENCIDA` y sube de prioridad.
- Al resolver como `RESUELTA_MODIFICADA`, la acción pasa a `MODIFICADA` y **devuelves los puntos**.

Coordina con B: él tiene la función de recálculo. Tú la llamas, no la reescribes.

D16, P8 — Citas

- El docente publica su horario; el sistema lo parte en bloques de **15 minutos**.
- El representante reserva un bloque; nace `SOLICITADA`.
- **Requiere confirmación del docente** para pasar a `CONFIRMADA`.

Éxito de la semana 5: un representante reclama una acción, el docente la resuelve como modificada, y el puntaje del estudiante sube.

Semana 6 · 13 al 19 de septiembre — Cierre

Trabajo conjunto. Tú eres responsable de:

- **Que RevenueCat funcione en el video de demostración.** Es lo primero que va a revisar el jurado.
- El build final firmado.
- Verificar el README en una máquina limpia.
- Que el repositorio quede público con `LICENSE` visible en la sección "About" de GitHub.
- Que el envío no se deje para el último día — meta interna 20 de septiembre, envío real 28. Solo quedan 8 días entre las dos.

Parte 6 — Trampas y consejos

Ya no hace falta empezar Play Console la primera semana. Ese fue el consejo principal de la v1.0 de este manual y quedó obsoleto: Next Gen no exige tienda. Tu prioridad #1 real sigue siendo que RevenueCat funcione, demostrablemente, en un teléfono real — solo que ya no necesitas una cuenta de desarrollador para lograrlo.

No escribas precios en el código. Vienen de `getOfferings()`. Si los escribes a mano, se desincronizarán de la tienda y mostrarás precios falsos.

El webhook debe ser idempotente desde el primer día. No es una optimización posterior: RevenueCat reenvía eventos de forma normal, y sin idempotencia tendrás suscripciones duplicadas que son difíciles de limpiar.

Verifica límites en el servidor. La app puede modificarse. El servidor no.

Prueba las compras en un teléfono real. El emulador no tiene Google Play Services completo y las compras fallan de formas que confunden.

La alerta de emergencia es delicada. Una falsa alarma en un colegio real genera pánico. Confirmación de dos pasos, registro de quién la activó, y la declaración visible de que no sustituye al ECU 911.

Guarda las claves en `.env`, nunca en el código. Y confirma que `.env` está en `.gitignore` antes de tu primer commit — el repositorio será público antes de enviar.

Parte 7 — Cuando te bloques

1. **Documentación oficial primero.** RevenueCat y Expo cambian rápido; la documentación oficial está más actualizada que cualquier tutorial.
2. **Relee el contrato** en `api/openapi.yaml` (v1.1.0).
3. **Medio día máximo.** Si llevas más, avisa al grupo ese mismo día. Si el bloqueo es de RevenueCat, avisa **el mismo día**, no medio: es riesgo del proyecto entero, no solo tuyo.
4. **Pregunta a una IA** pegando el bloque de la siguiente sección.

Parte 8 — Bloque de contexto para IA

Copia todo esto y pégalo antes de tu pregunta.

CONTEXTO – PROYECTO CRESCO, ROL: PERSONA C (INTERACCIÓN, PAGOS E INFRA)

Soy uno de 3 estudiantes (equipo Neofix) construyendo una app Android para el hackathon RevenueCat Shipaton 2026, categoría Next Gen. Semana 1 desde el 9 de agosto de 2026; deadline interno 20 de septiembre; envío real 28 de septiembre. Es mi primera app. Mi rol cubre citas, inconformidades, alertas de emergencia, RevenueCat, notificaciones push, CI y despliegue.

SOY YO QUIEN SCAFFOLDEA servidor/ (Next, npx create-next-app) y movil/ (Expo, npx create-expo-app) el día 1 de la semana 1 – decisión S6. Mis endpoints viven en servidor/, mis pantallas en movil/. Es una sola app Expo con enrutado por rol, no dos apps. Sin npm workspaces entre las dos: cada una tiene su propio package.json.

EL PRODUCTO: app de comunicación entre docentes y representantes legales de escuelas del Ecuador, para seguimiento de conducta. El estudiante NUNCA es usuario. Dos interfaces: docente y representante.

STACK: PostgreSQL + Drizzle ORM. Next.js como servidor (BetterAuth, rutas API, webhook de RevenueCat, tarea nocturna). Expo/React Native para las apps. NativeWind para estilos (shadcn NO funciona en React Native).

MIS TABLAS: disponibilidad_docente, cita, inconformidad, alerta_emergencia, entrega_alerta, plan, suscripcion, evento_revenuecat, desbloqueo_recompensado, dispositivo, notificacion, auditoria.

REVENUECAT (requisito OBLIGATORIO del hackathon):

- NO hace falta cuenta de Google Play Console ni publicación en tienda para el envío: Next Gen se evalúa por video y repositorio abierto. Pendiente: confirmar el modo de prueba de compras sin tienda en la documentación oficial de RevenueCat.
- RevenueCat es la FUENTE DE VERDAD del estado de suscripción; la tabla suscripcion es solo una proyección local.
- El webhook debe ser IDEMPOTENTE: evento_revenuecat.evento_id_externo tiene índice único; un reenvío no duplica nada.
- Entitlement "premium". 5 planes: REP_FREE, REP_PREMIUM_MENSUAL,

REP_PREMIUM_BIMESTRAL, DOC_FREE, DOC_PRO.

- "Por parcial" se mapea a suscripción BIMESTRAL (Google Play no tiene ciclo de 6 semanas).
- Límites en plan.limite como JSON: DOC_FREE = 1 curso activo y 40 estudiantes; DOC_PRO = 5 cursos y 60 estudiantes. REP_FREE = 2 reportes anteriores; premium = 7 y sin publicidad.
- Los límites se verifican EN EL SERVIDOR, no en la app. Al alcanzarlos el endpoint devuelve 402 con código LIMITE_PLAN y la app muestra el paywall.
- RevenueCat en Expo requiere DEVELOPMENT BUILD; no funciona en Expo Go.

REGLAS DE MI MÓDULO:

- ALERTAS: el docente debe REINGRESAR SU CONTRASEÑA antes de activar (se guarda reautenticado_en). Alcance CURSO o ESTUDIANTE. Queda auditada. La app debe declarar que NO sustituye al ECU 911. El representante confirma lectura.
- INCONFORMIDADES: solo sobre acciones NEGATIVAS vigentes. El docente tiene 30 días; vencido pasa a VENCIDA. Resolver como RESUELTA_MODIFICADA lleva la acción a MODIFICADA (0 puntos) y DEVUELVE los puntos llamando a la función de recálculo de Persona B.
- CITAS: el docente publica horario, se parte en bloques de 15 minutos. El representante reserva y nace SOLICITADA; REQUIERE confirmación del docente.
- El plan gratuito puede exportar el PDF del acumulado viendo un anuncio recompensado (tabla desbloqueo_recompensado). El endpoint devuelve el PDF directo en la respuesta, sin URL ni almacenamiento (no se guardan archivos en ninguna parte).
- Licencia del repositorio: AGPL-3.0 con posibilidad de licenciamiento comercial en paralelo.

SEGURIDAD: prohibido usar db.select() suelto sobre estudiante, matricula, accion_registrada o reporte_estudiante. Toda consulta pasa por db/acceso/.

CONVENCIONES: tablas y columnas en español snake_case. Estados en MAYUSCULA_SNAKE en db/schema/enums.ts. Timestamps timestampz en UTC, zona America/Guayaquil. Claves y secretos siempre en .env, nunca en el código – el repositorio será público antes de enviar.

MI RESTRICCIÓN: 6 semanas, primera app, prefiero soluciones simples y probadas. Explicame los conceptos, no solo el código.